

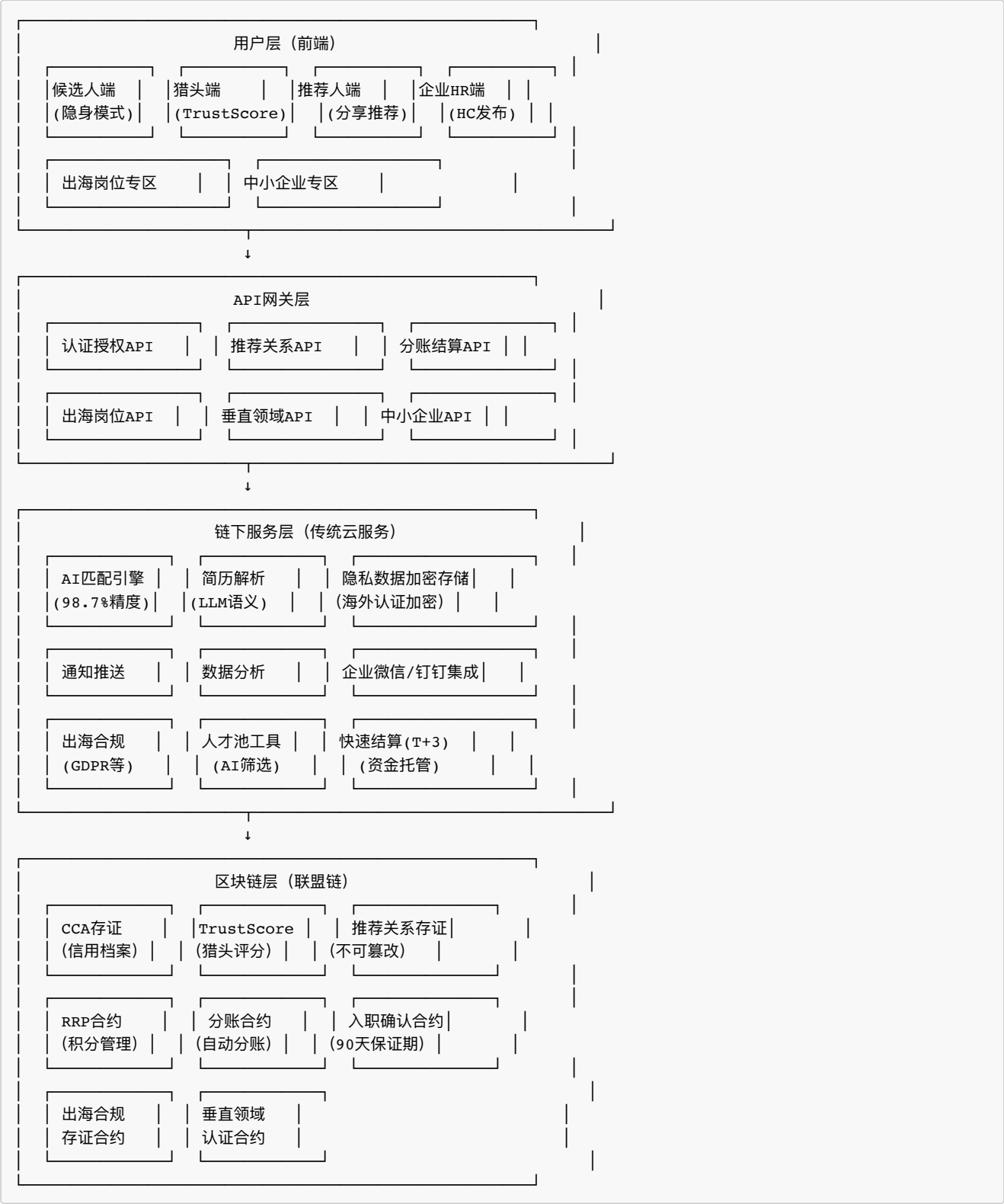
人人猎（RenrenLie）技术说明文档

版本：v1.8 | 日期：2026年4月30日

一、技术架构概述

1.1 总体架构设计

人人猎采用链上+链下混合架构，将核心信任数据上链存证，业务数据链下处理，兼顾性能与可信性。

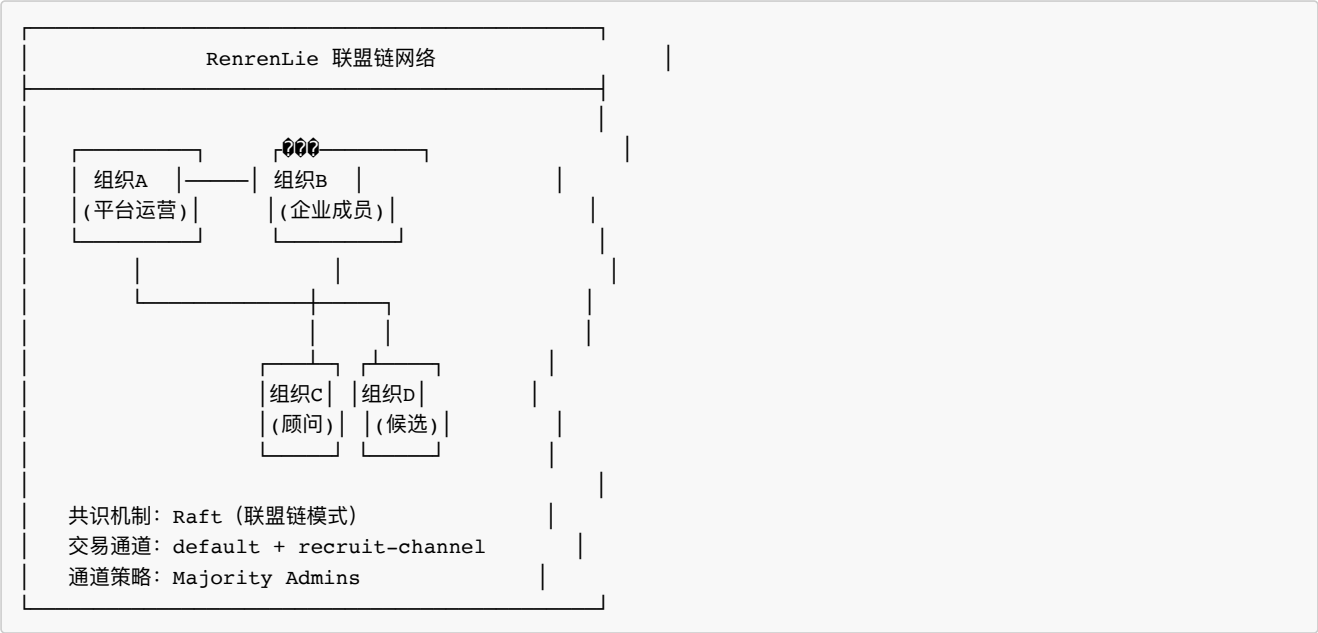


1.2 技术栈选型

层级	技术选型	说明
前端	React 18 + TypeScript + Tailwind CSS	响应式设计，支持PC/移动端
后端API	Node.js + Express + TypeScript	RESTful API, JWT认证
区块链	Hyperledger Fabric（联盟链）	企业级许可链，性能高，隐私性好
数据库	PostgreSQL + Redis	链下数据缓存，提高查询性能
AI匹配	Python + LangChain + LLM	JD解析、简历匹配、圈层分析
存储	IPFS（链下大文件）+ 阿里云OSS	简历附件、作品集等
消息队列	Kafka	异步处理链上事件
结算服务	Go	T+3快速结算、资金托管、高并发分账

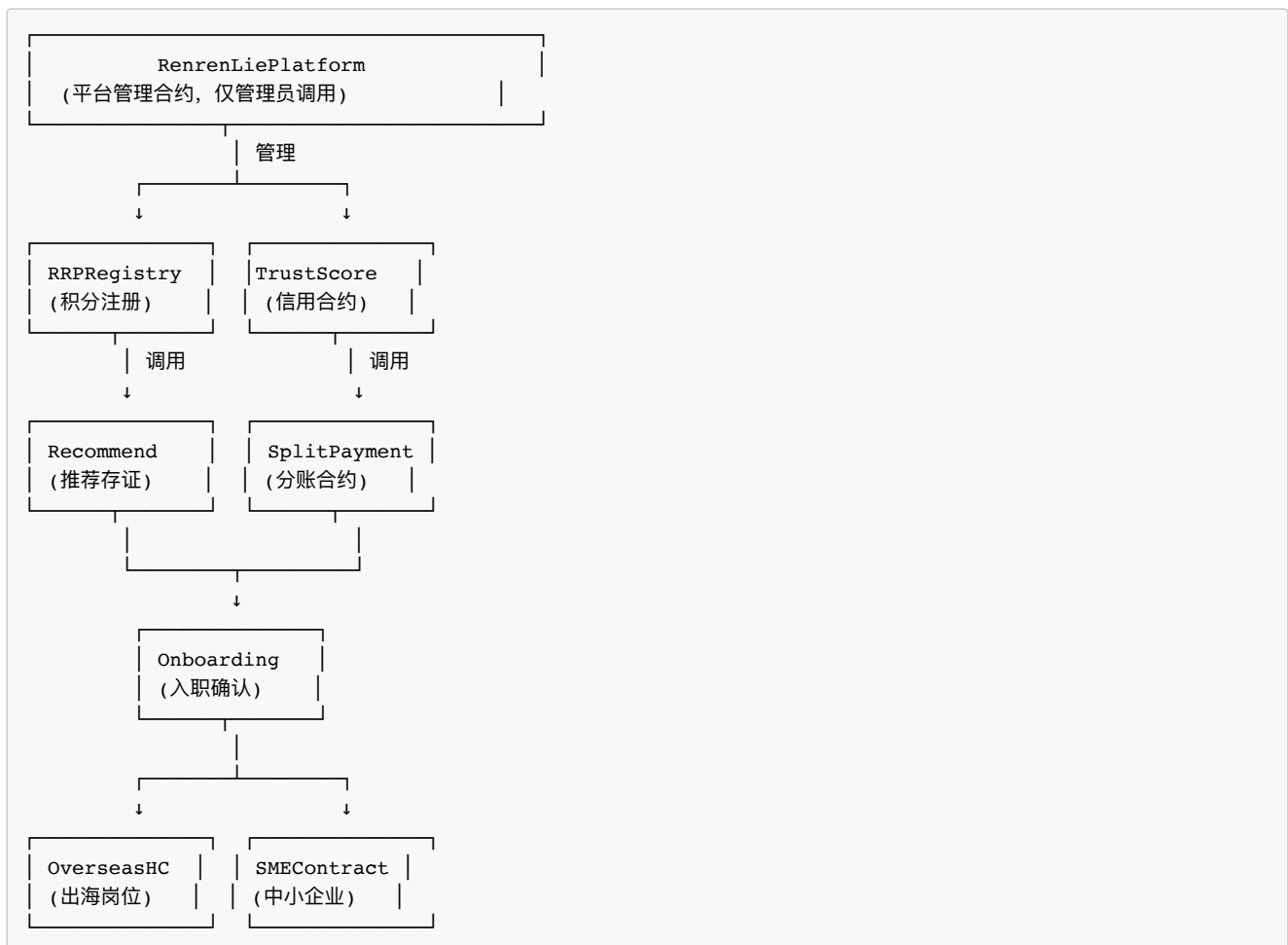
二、区块链架构设计

2.1 联盟链网络架构



2.2 智能合约架构

2.2.1 合约层次结构



2.2.2 核心合约代码

1. 推荐存证合约（Recommend.sol）

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

/**
 * @title Recommend - 推荐关系存证合约
 * @dev 记录猎头与候选人的推荐关系，不可篡改
 */
contract Recommend {
    struct Recommendation {
        bytes32 id;           // 推荐ID
        address hunter;       // 猎头钱包地址
        address candidate;    // 候选人钱包地址
        uint256 jobId;        // 职位ID
        uint256 timestamp;    // 推荐时间戳
        Status status;        // 推荐状态
        string statusReason;  // 状态变更原因
    }

    enum Status {
        Pending,           // 待确认
        Interviewing,      // 面试中
        Hired,              // 已录用
        Onboarding,        // 入职中
        Passed,             // 已入职（通过保证期）
        Failed              // 失败
    }
}
```

```

mapping(bytes32 => Recommendation) public recommendations;
mapping(address => bytes32[]) public hunterRecommendations;
mapping(address => bytes32[]) public candidateRecommendations;

event RecommendationCreated(
    bytes32 indexed id,
    address indexed hunter,
    address indexed candidate,
    uint256 jobId
);

event StatusChanged(
    bytes32 indexed id,
    Status oldStatus,
    Status newStatus,
    string reason
);

/**
 * @dev 创建推荐存证
 */
function createRecommend(
    address _hunter,
    address _candidate,
    uint256 _jobId
) external returns (bytes32) {
    bytes32 id = keccak256(
        abi.encodePacked(_hunter, _candidate, _jobId, block.timestamp)
    );

    recommendations[id] = Recommendation({
        id: id,
        hunter: _hunter,
        candidate: _candidate,
        jobId: _jobId,
        timestamp: block.timestamp,
        status: Status.Pending,
        statusReason: "Created"
    });

    hunterRecommendations[_hunter].push(id);
    candidateRecommendations[_candidate].push(id);

    emit RecommendationCreated(id, _hunter, _candidate, _jobId);
    return id;
}

/**
 * @dev 更新推荐状态
 */
function updateStatus(
    bytes32 _id,
    Status _newStatus,
    string calldata _reason
) external {
    Recommendation storage rec = recommendations[_id];
    require(rec.hunter != address(0), "Recommendation not found");

    Status oldStatus = rec.status;
    rec.status = _newStatus;
    rec.statusReason = _reason;

    emit StatusChanged(_id, oldStatus, _newStatus, _reason);
}

```

```

/**
 * @dev 获取推荐详情
 */
function getRecommend(bytes32 _id) external view returns (
    address hunter,
    address candidate,
    uint256 jobId,
    uint256 timestamp,
    Status status,
    string memory statusReason
) {
    Recommendation storage rec = recommendations[_id];
    return (
        rec.hunter,
        rec.candidate,
        rec.jobId,
        rec.timestamp,
        rec.status,
        rec.statusReason
    );
}
}

```

2. 分账合约 (SplitPayment.sol)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

/**
 * @title SplitPayment - 智能合约分账
 * @dev 管理猎头、候选人、平台的三方分账，支持T+3快速结算
 * @dev v1.8支持：出海岗位高佣金、中小企业差异化分账
 */
contract SplitPayment {
    address public admin;
    address public platformWallet;

    // 出海岗位标识
    mapping(uint256 => bool) public isOverseasJob;
    // 中小企业标识
    mapping(uint256 => bool) public isSMEJob;

    event PaymentReleased(
        uint256 jobId,
        address hunter,
        uint256 hunterAmount,
        address candidate,
        uint256 candidateAmount,
        uint256 platformAmount,
        bool isOverseas,
        bool isSME
    );

    modifier onlyAdmin() {
        require(msg.sender == admin, "Not admin");
        _;
    }

    constructor(address _platformWallet) {
        admin = msg.sender;
        platformWallet = _platformWallet;
    }
}

```

```

/**
 * @dev 释放佣金（支持出海/中小企业差异化分账）
 */
function releasePayment(
    uint256 jobId,
    address hunter,
    address candidate,
    uint256 totalAmount
) external onlyAdmin {
    require(totalAmount > 0, "Invalid amount");

    uint256 hunterShare;
    uint256 candidateShare;
    uint256 platformShare;

    // 根据岗位类型调整分账比例
    if (isOverseasJob[jobId]) {
        // 出海岗位：猎头85% / 候选人15% / 平台0%
        hunterShare = (totalAmount * 85) / 100;
        candidateShare = (totalAmount * 15) / 100;
        platformShare = 0;
    } else if (isSMEJob[jobId]) {
        // 中小企业：猎头70% / 候选人15% / 平台15%
        hunterShare = (totalAmount * 70) / 100;
        candidateShare = (totalAmount * 15) / 100;
        platformShare = (totalAmount * 15) / 100;
    } else {
        // 标准岗位：猎头80% / 候选人15% / 平台5%
        hunterShare = (totalAmount * 80) / 100;
        candidateShare = (totalAmount * 15) / 100;
        platformShare = (totalAmount * 5) / 100;
    }

    emit PaymentReleased(
        jobId,
        hunter,
        hunterShare,
        candidate,
        candidateShare,
        platformShare,
        isOverseasJob[jobId],
        isSMEJob[jobId]
    );
}

/**
 * @dev 设置出海岗位标识
 */
function setOverseasJob(uint256 jobId, bool status) external onlyAdmin {
    isOverseasJob[jobId] = status;
}

/**
 * @dev 设置中小企业标识
 */
function setSMEJob(uint256 jobId, bool status) external onlyAdmin {
    isSMEJob[jobId] = status;
}
}

```

3. TrustScore合约 (TrustScore.sol)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

```

```

/**
 * @title TrustScore - 猎头信用评分合约
 * @dev 记录猎头TrustScore, 不可篡改
 * @dev v1.8支持: 垂直领域加分
 */
contract TrustScore {
    struct HunterScore {
        uint256 baseScore;           // 基础分 (50分)
        uint256 successBonus;        // 成单加分 (最高25分)
        uint256 ratingBonus;         // 评价加分 (最高12分)
        uint256 creditBonus;         // 信用加分 (最高5分)
        uint256 verticalBonus;       // 垂直领域加分 (最高8分)
        int256 violationPenalty;     // 违规扣分 (不封顶)
        uint256 totalScore;          // 总分
        uint256 lastUpdate;          // 最后更新时间
    }

    mapping(address => HunterScore) public hunterScores;
    mapping(address => bool) public isVerifiedHunter;
    mapping(address => mapping(uint256 => bool)) public verticalDomains; // 垂直领域认证

    event HunterRegistered(address indexed hunter);
    event ScoreUpdated(
        address indexed hunter,
        uint256 oldScore,
        uint256 newScore,
        string reason
    );
    event VerticalCertified(
        address indexed hunter,
        uint256 domain,
        bool status
    );

    /**
     * @dev 注册猎头
     */
    function registerHunter(address _hunter) external {
        require(!isVerifiedHunter[_hunter], "Already registered");
        isVerifiedHunter[_hunter] = true;
        hunterScores[_hunter] = HunterScore({
            baseScore: 50,
            successBonus: 0,
            ratingBonus: 0,
            creditBonus: 0,
            verticalBonus: 0,
            violationPenalty: 0,
            totalScore: 50,
            lastUpdate: block.timestamp
        });
        emit HunterRegistered(_hunter);
    }

    /**
     * @dev 更新TrustScore
     */
    function updateScore(
        address _hunter,
        int256 _delta,
        string calldata _reason
    ) external {
        require(isVerifiedHunter[_hunter], "Hunter not registered");

```

```

HunterScore storage score = hunterScores[_hunter];
uint256 oldScore = score.totalScore;

if (_delta > 0) {
    // 加分
    if (score.totalScore + uint256(_delta) > 100) {
        score.totalScore = 100;
    } else {
        score.totalScore += uint256(_delta);
    }
} else {
    // 扣分
    if (score.totalScore < uint256(-_delta)) {
        score.totalScore = 0;
        score.violationPenalty += _delta;
    } else {
        score.totalScore -= uint256(-_delta);
        score.violationPenalty += _delta;
    }
}

score.lastUpdate = block.timestamp;
emit ScoreUpdated(_hunter, oldScore, score.totalScore, _reason);
}

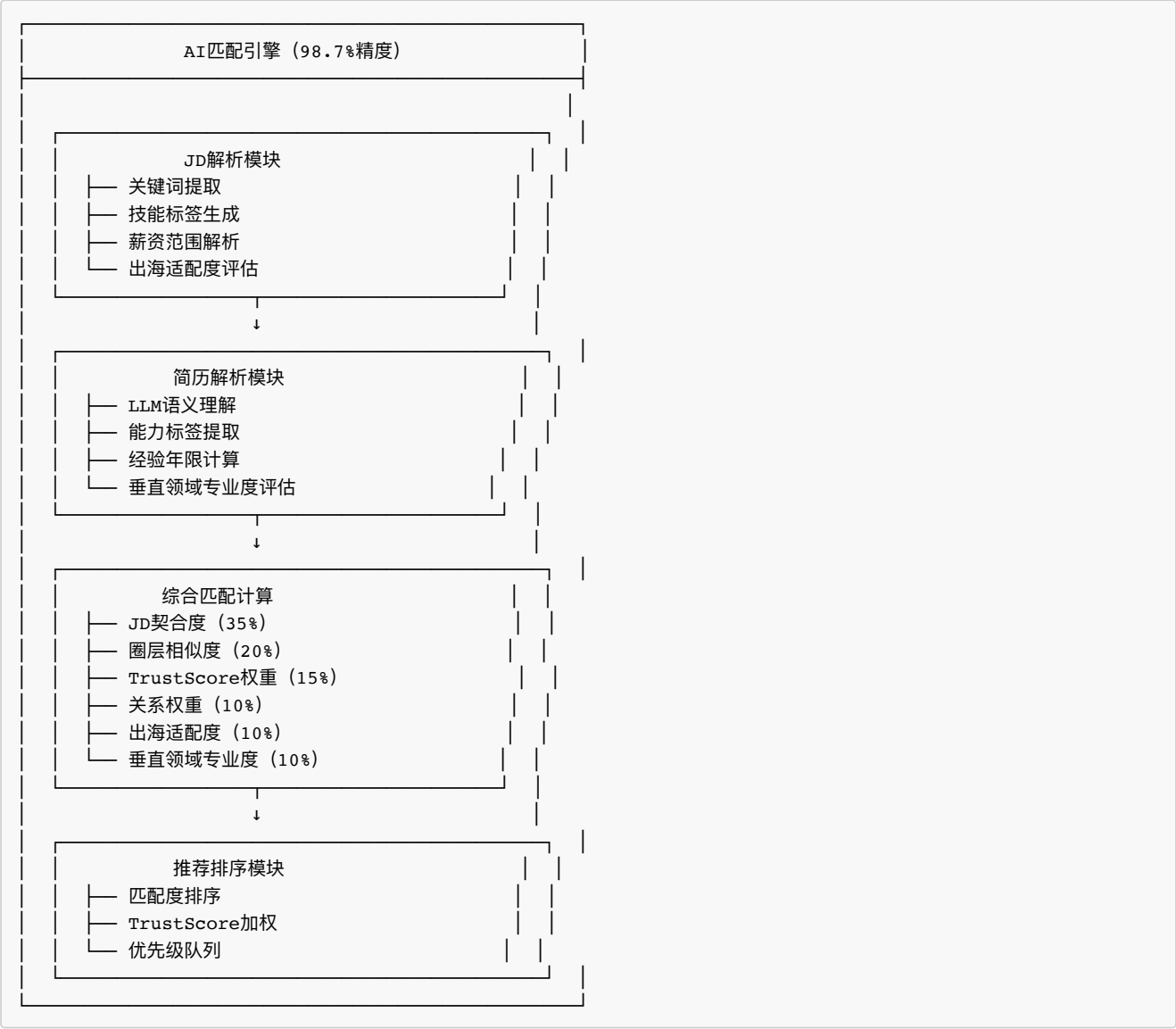
/**
 * @dev 添加垂直领域认证
 */
function certifyVerticalDomain(
    address _hunter,
    uint256 _domain,
    bool _status
) external {
    require(isVerifiedHunter[_hunter], "Hunter not registered");
    verticalDomains[_hunter][_domain] = _status;
    emit VerticalCertified(_hunter, _domain, _status);
}

/**
 * @dev 获取TrustScore
 */
function getScore(address _hunter) external view returns (uint256) {
    return hunterScores[_hunter].totalScore;
}
}

```

三、AI匹配引擎设计

3.1 匹配引擎架构



3.2 综合匹配度算法

综合匹配度 = JD契合度 × 0.35
+ 圈层相似度 × 0.20
+ TrustScore权重 × 0.15
+ 关系权重 × 0.10
+ 出海适配度 × 0.10
+ 垂直领域专业度 × 0.10

取值范围：0-100分
推荐阈值：≥60分 → 系统自动推送给猎头
优质阈值：≥80分 → 系统优先推荐给高TrustScore猎头
出海岗位阈值：≥70分 → 专项推送给出海认证猎头

各维度详细计算

1. JD契合度（权重35%）

JD契合度 = 技能匹配度 × 0.3
+ 经验匹配度 × 0.25
+ 薪资匹配度 × 0.2
+ 行业匹配度 × 0.15
+ 出海适配度 × 0.1

基于LLM语义理解，精度98.7%。

2. 圈层相似度（权重20%）

圈层相似度 = $\Sigma(\text{圈层权重} \times \text{圈层重叠度})$

圈层权重：

- 大厂圈层：1.0（腾讯/字节/阿里/美团/拼多多）
- 行业圈层：0.7（互联网/金融/医疗/教育）
- 校友圈层：0.5（985/211/海外名校）
- 垂直领域圈层：0.9（AI社群/芯片论坛/新能源协会）

3. TrustScore权重（权重15%）

TrustScore权重 = $\text{推荐猎头TrustScore} / 100$

高TrustScore猎头推荐的候选人，匹配可信度更高。

4. 关系权重（权重10%）

关系权重 = $\text{关系类型权重} \times \text{关系强度}$

关系类型权重：

- 一度人脉：1.0
- 二度人脉：0.7
- 三度人脉：0.3

5. 出海适配度（权重10%）

出海适配度 = $\text{海外经历评分} \times 0.4$
+ $\text{语言能力评分} \times 0.3$
+ $\text{跨文化适配度} \times 0.2$
+ $\text{技术匹配度} \times 0.1$

6. 垂直领域专业度（权重10%）

垂直领域专业度 = $\text{领域经验评分} \times 0.4$
+ $\text{技术认证评分} \times 0.3$
+ $\text{项目成果评分} \times 0.2$
+ $\text{行业影响力} \times 0.1$

3.3 匹配度应用场景

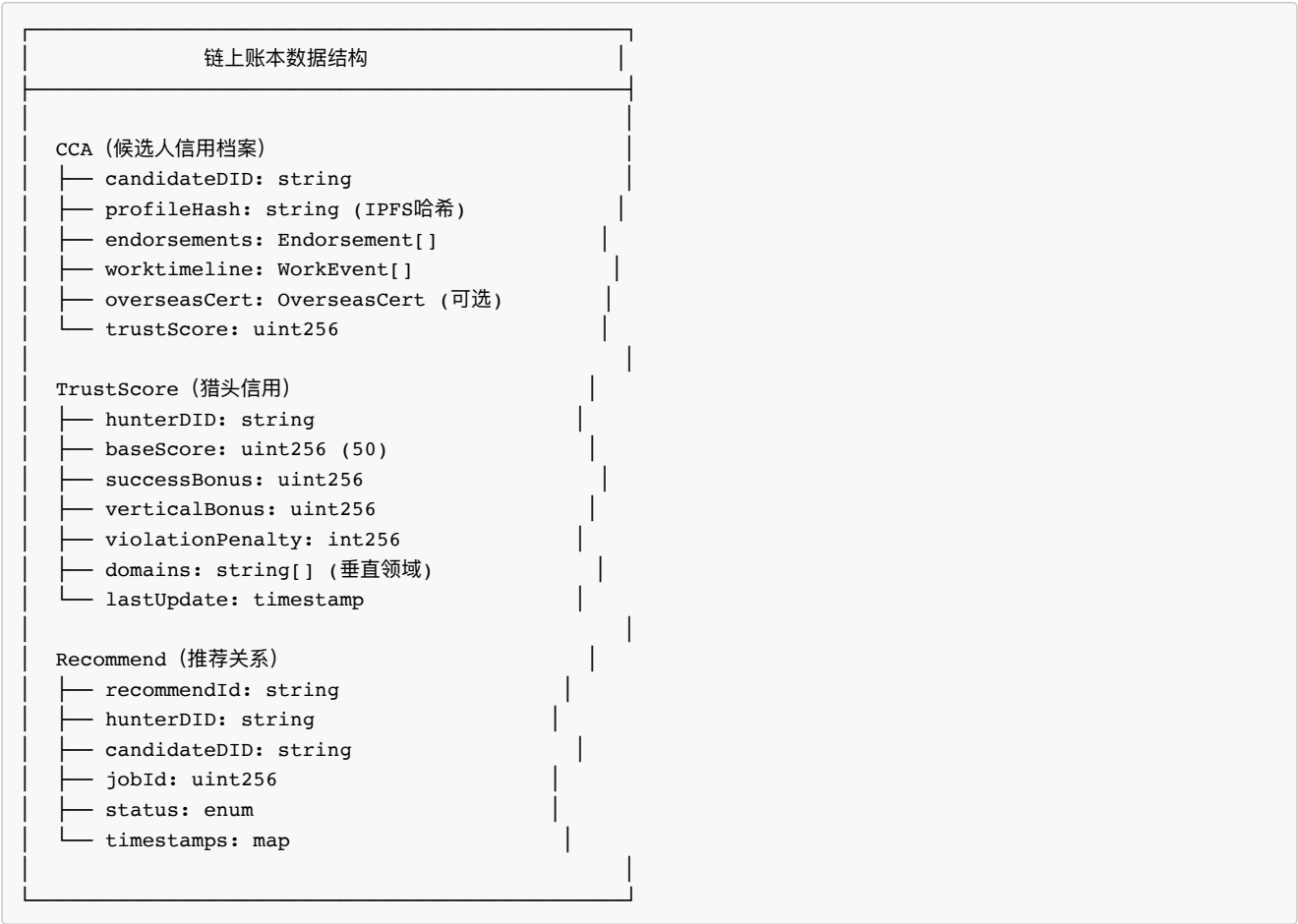
场景	阈值	推送规则
猎头HC推荐	≥60分	系统自动推送给猎头
候选人职位推荐	≥60分	系统推送给候选人
优先推荐	≥80分	优先推送给高TrustScore猎头
出海岗位推荐	≥70分	推送给出海认证猎头
垂直领域推荐	≥75分	推送给垂直领域认证猎头
中小企业快速匹配	≥60分	48小时内响应



四、数据架构设计

4.1 数据模型

4.1.1 链上数据 (Hyperledger Fabric)



4.1.2 链下数据 (PostgreSQL)

```
-- 用户表
CREATE TABLE users (
  id UUID PRIMARY KEY,
  one_id VARCHAR(64) UNIQUE NOT NULL,
  role VARCHAR(20) NOT NULL, -- 'candidate', 'hunter', 'referrer', 'hr'
  wallet_address VARCHAR(64),
  phone VARCHAR(20),
  email VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- 候选人详情表
CREATE TABLE candidate_profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id),
  resume_data JSONB, -- 简历完整数据
  overseas_flags JSONB, -- 海外认证标签
  vertical_domains JSONB, -- 垂直领域标签
  privacy_settings JSONB, -- 隐私设置
  is_invisible BOOLEAN DEFAULT TRUE, -- 隐身模式
  created_at TIMESTAMP DEFAULT NOW()
);

-- 猎头详情表
CREATE TABLE hunter_profiles (
```

```

    user_id UUID PRIMARY KEY REFERENCES users(id),
    company_name VARCHAR(100),
    license_number VARCHAR(50),
    vertical_certifications JSONB, -- 垂直领域认证
    overseas_certifications JSONB, -- 出海岗位认证
    trust_score DECIMAL(5,2) DEFAULT 50.00,
    total_recommendations INTEGER DEFAULT 0,
    successful_hires INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- 职位表
CREATE TABLE jobs (
    id UUID PRIMARY KEY,
    company_id UUID REFERENCES users(id),
    title VARCHAR(200) NOT NULL,
    description TEXT,
    requirements JSONB, -- JD解析结果
    salary_range JSONB, -- 薪资范围
    is_overseas BOOLEAN DEFAULT FALSE, -- 出海岗位
    is_sme BOOLEAN DEFAULT FALSE, -- 中小企业
    vertical_domain VARCHAR(50), -- 垂直领域
    status VARCHAR(20) DEFAULT 'open',
    created_at TIMESTAMP DEFAULT NOW()
);

-- 推荐关系表
CREATE TABLE recommendations (
    id UUID PRIMARY KEY,
    hunter_id UUID REFERENCES users(id),
    candidate_id UUID REFERENCES users(id),
    job_id UUID REFERENCES jobs(id),
    chain_recommend_id VARCHAR(64), -- 链上存证ID
    status VARCHAR(20) DEFAULT 'pending',
    match_score DECIMAL(5,2),
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

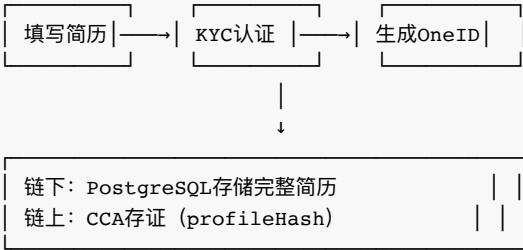
-- 面试反馈表
CREATE TABLE interview_feedback (
    id UUID PRIMARY KEY,
    recommendation_id UUID REFERENCES recommendations(id),
    rating INTEGER CHECK (rating >= 1 AND rating <= 5),
    feedback_text TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

```

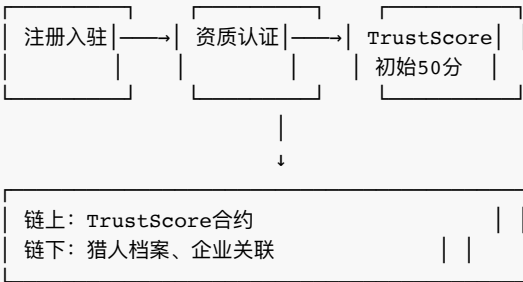
4.2 数据流转

数据流转设计

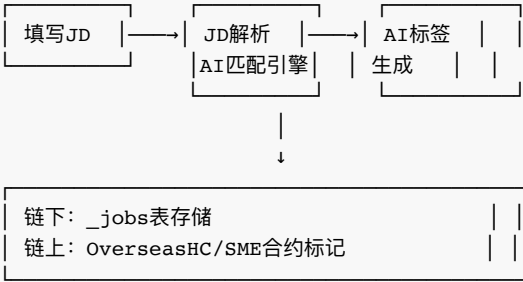
候选人注册



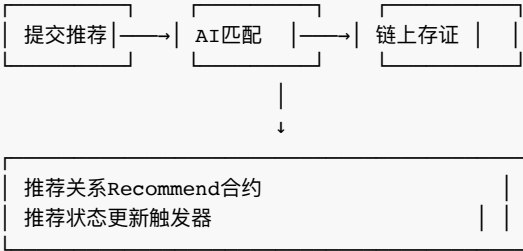
猎头入驻



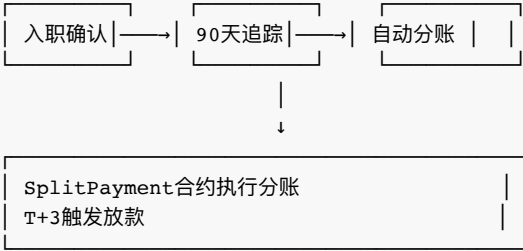
发布职位



推荐候选人



入职分账



五、API架🔗🔗🔗设🔗🔗🔗

5.1 API列表

5.1.1 认证API

方法	路径	说明
POST	/api/auth/register	用户注册
POST	/api/auth/login	登录
POST	/api/auth/kyc	KYC认证
GET	/api/auth/oneid/:address	查询OneID

5.1.2 用户API

方法	路径	说明
GET	/api/users/me	获取当前用户信息
PUT	/api/users/me	更新用户信息
GET	/api/users/:id	获取其他用户公开信息
PUT	/api/users/privacy	隐私设置

5.1.3 候选人API

方法	路径	说明
POST	/api/candidate/resume	创建简历
GET	/api/candidate/resume	获取简历
PUT	/api/candidate/resume	更新简历
PUT	/api/candidate/invisible	设置隐身模式
POST	/api/candidate/authorize	授权猎头查看
GET	/api/candidate/cca	获取CCA信用档案

5.1.4 猎头API

方法	路径	说明
POST	/api/hunter/profile	创建猎头档案
GET	/api/hunter/profile	获取猎头档案
GET	/api/hunter/trustscore	获取TrustScore
GET	/api/hunter/candidates	浏览候选人池
POST	/api/hunter/recommend	推荐候选人
GET	/api/hunter/recommendations	推荐列表
PUT	/api/hunter/status	更新推荐状态

5.1.5 职位API

方法	路径	说明
POST	/api/jobs	发布职位
GET	/api/jobs	职位列表
GET	/api/jobs/:id	职位详情
PUT	/api/jobs/:id	更新职位
DELETE	/api/jobs/:id	删除职位
GET	/api/jobs/overseas	出海职位专区
GET	/api/jobs/sme	中小企业专区

5.1.6 分账API

方法	路径	说明
POST	/api/payment/onboarding	入职确认
GET	/api/payment/guarantee/:jobId	保证期状态
POST	/api/payment/release	触发分账
GET	/api/payment/history	分账历史

5.2 API响应格式

```
// 成功响应
{
  "success": true,
  "data": {
    "id": "xxx",
    "one_id": "did:renrenlie:xxx",
    "trust_score": 65
  },
  "message": "success"
}

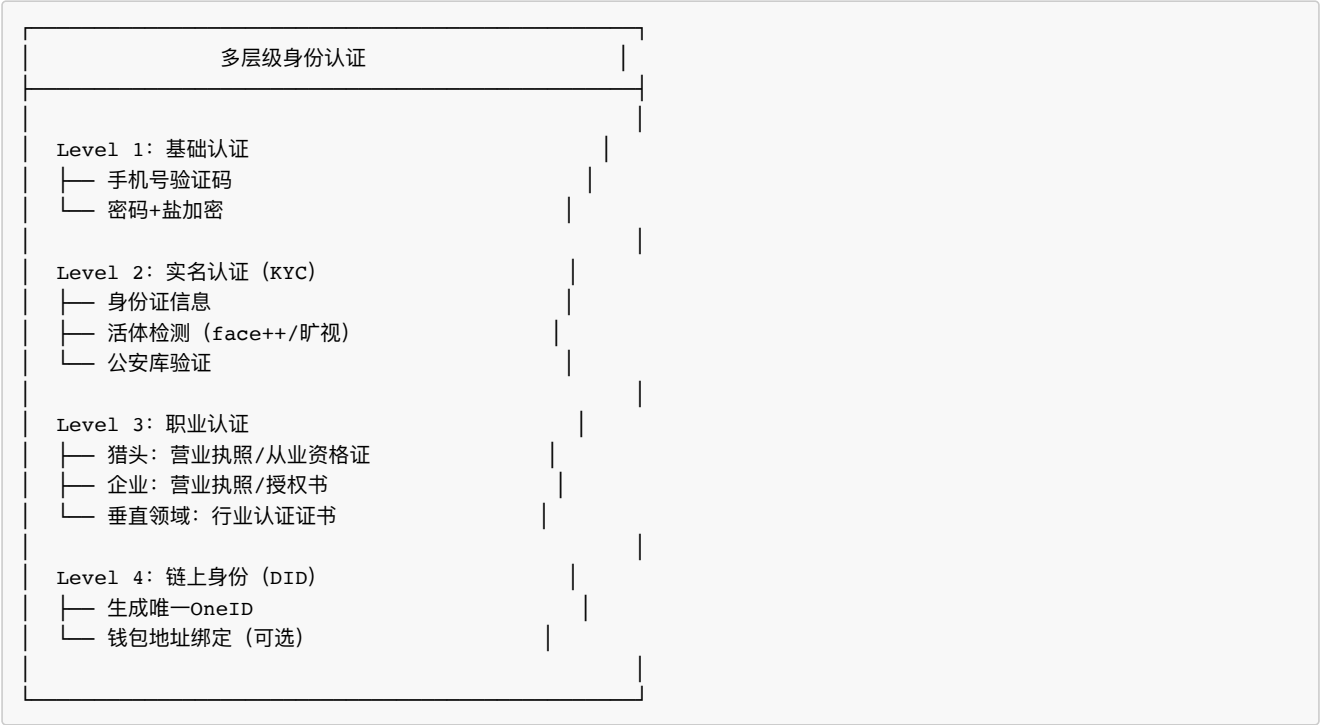
// 错误响应
{
  "success": false,
  "error": {
    "code": "INVALID_PARAMS",
    "message": "参数错误"
  }
}

// 分页响应
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 1,
    "page_size": 20,
    "total": 100
  }
}
```



六、全设计

6.1 身份认证



6.2 数据安全

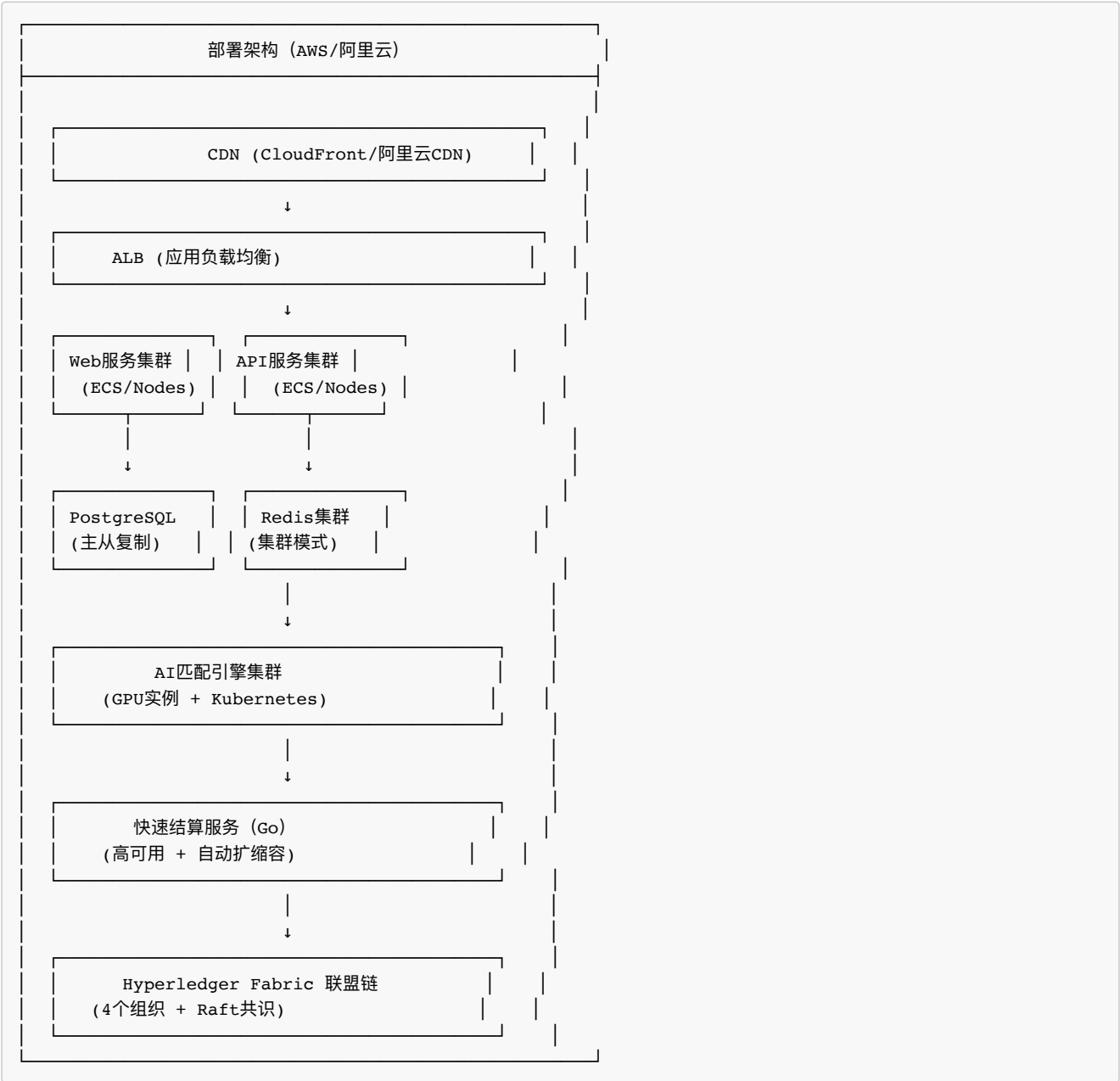
安全措施	说明
传输加密	全站HTTPS/TLS 1.3
存储加密	链下数据AES-256加密
隐私保护	候选人默认隐身模式
授权机制	候选人主动授权解锁
数据隔离	链上/链下数据分离
ZKP	零知识证明 (可选)

6.3 智能合约安全

安全措施	说明
代码审计	CertiK/慢雾第三方审计
权限控制	onlyAdmin修饰符
溢出保护	OpenZeppelin SafeMath
紧急暂停	Pause功能
漏洞赏金	Immunefi漏洞赏金

七、部署架构设计

7.1 基础设施架构



7.2 高可用设计

组件	高可用策略
Web/API	ALB + 多AZ部署 + Auto Scaling
数据库	主从复制 + 自动故障切换
Redis	Cluster模式 + Sentinel
AI引擎	GPU池化 + 负载均衡
结算服务	多副本 + 消息队列削峰
区块链	联盟链多组织共识

7.3 监控告警

监控项	指标
服务可用性	SLO 99.9%
API响应时间	P99 < 500ms
区块链TPS	> 1000 tx/s
AI匹配精度	> 98.7%
结算成功率	> 95%
错误率	< 0.1%

八、TrustScore算法详解

8.1 数据来源

- 1. 猎头身份认证 - 实名认证（手机号+身份证+活体检测） - 从业资质认证（营业执照/从业资格证） - 社交账号绑定（微信/LinkedIn）
- 垂直领域认证（AI/芯片/新能源等）
- 2. 圈层数据 - 候选人社交图谱分析 - 行业圈层标签（腾讯系/字节系/阿里系） - 职业社交影响力 - 垂直领域圈层
- 3. 推荐历史存证 - 链上推荐记录（推荐ID、猎头ID、候选人ID、职位ID、时间戳） - 推荐结果（面试/录用/入职/失败） - 企业反馈（评分+文字评价） - 垂直领域匹配标记
- 4. 成单率与质量 - 成单率 = 入职数 / 推荐总数 - 平均薪资涨幅 - 保证期通过率（90天） - 出海岗位成单率
- 5. CCA信用数据 - 职业时⚡️线 - 企业背书 - 候选人隐身授权记录
- 6. 垂直领域专业度 - 垂直领域认证 - 领域内成单率 - 领域企业评价

8.2 计算公式

TrustScore = 基础分(50) + 成单加分(25) + 评价加分(12) + 信用加分(5) + 垂直领域加分(8) - 违规扣分(不封顶)

1. 基础分(50分): 完成实名认证+从业资质认证 → 自动获得

2. 成单加分(最高25分):

- 每成功入职1人 → +2.5分 (最高25分)
- 保证期通过率≥90% → 额外+3分
- 平均薪资涨幅≥30% → 额外+2分
- 出海岗位成单 → 每单额外+3分 (最高15分)

3. 评价加分(最高12分):

- 企业HR评价: 每次面试反馈 +0.4分 (最高8分)
- 候选人评价: 每次授权解锁 +0.2分 (最高4分)

4. 信用加分(最高5分):

- 连续90天无违规 → +2分
- 推荐信息真实无造假 → +3分

5. 垂直领域加分(最高8分):

- 获得垂直领域认证 → +3分
- 领域内成单率≥30% → +3分
- 领域影响力 → +2分

6. 违规扣分(不封顶):

- 简历造假 → -20分/次
- 候选人投诉属实 → -10分/次
- 推荐后失联 → -5分/次
- 出海岗位虚假信息 → -30分/次

8.3 等级划分

分数段	等级	称号	权益
80-100	S级	金牌猎头	优先接高薪HC、流量倾斜
60-79	A级	优质猎头	正常推荐权限
40-59	B级	新人猎头	限制每日推荐次数
20-39	C级	观察期猎头	仅能推荐低薪岗位
<20	D级	黑名单	禁止推荐

九、关键业务流程

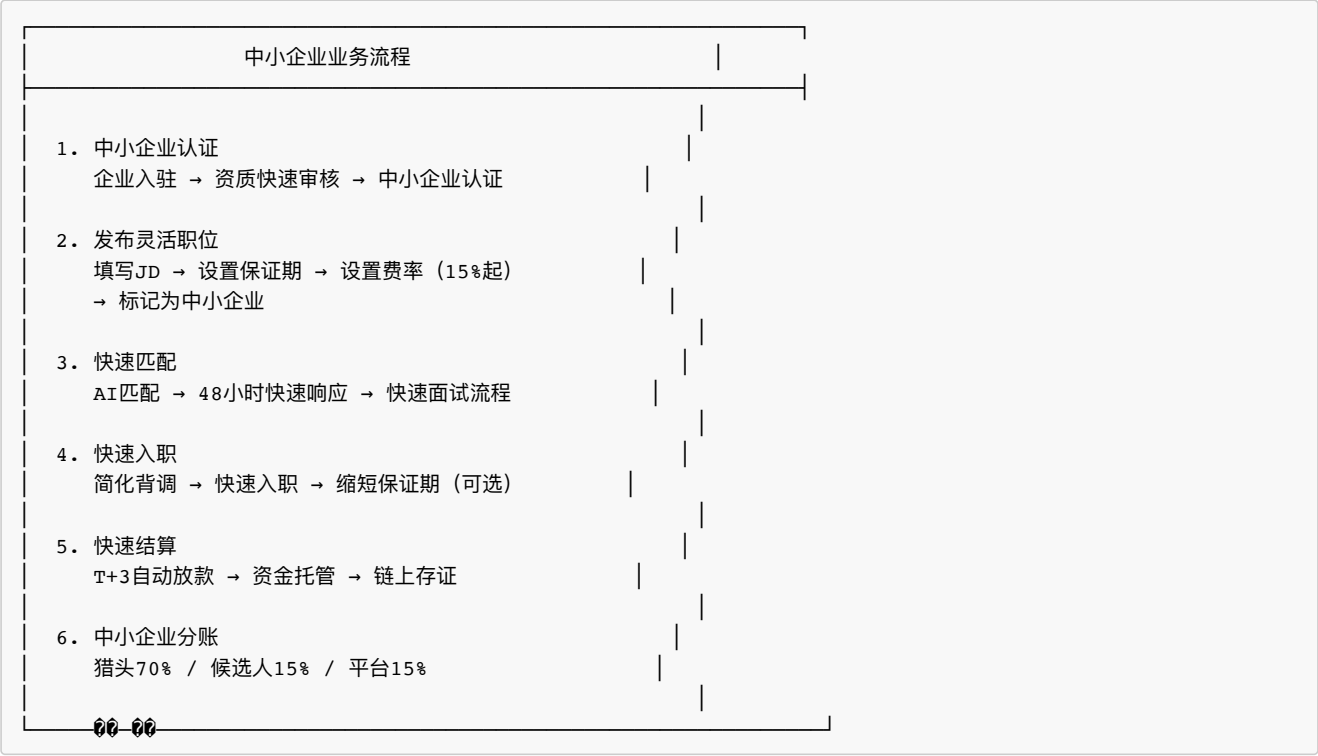
9.1 推荐流程

推荐业务流程	
1. 候选人入驻	注册 → KYC认证 → 生成OneID → 填写简历 → 设置隐身
2. 猎头发现候选人	浏览候选人池 → AI匹配推荐 → 查看候选人信息
3. 授权解锁	发起授权请求 → 候选人确认 → 授权成功
4. 推荐存证	提交推荐 → 链上存证 → 生成推荐ID
5. 面试流程	企业安排面试 → 猎头陪同 → 面试反馈 → 链上存证
6. 入职确认	候选人入职 → 企业确认 → 90天保证期开始
7. 分账结算	保证期结束 → 触发分账 → 智能合约自动分账

9.2 出海岗位流程

出海岗位业务流程	
1. 出海企业认证	企业入驻 → 海外资质审核 → 出海企业认证
2. 发布出海职位	填写JD → 选择海外工作地点 → 设置薪酬（3倍溢价） → 标记为出海岗位
3. 出海人才匹配	AI匹配 + 出海适配度评估 → 推送给出海认证猎头
4. 出海推荐	推荐存证 → 出海岗位标识 → 薪酬溢价标记
5. 跨文化面试	安排面试 → 跨文化适配评估 → 海外面试（视频）
6. 入职与海外到岗	入职确认 → 海外到岗 → 薪资发放
7. 出海分账	保证期结束 → 猎头85% / 候选人15% / 平台0%

9.3 中小企业流程



十、技术指标

10.1 性能指标

指标	目标值
API响应时间	P99 < 500ms
区块链确认时间	< 3秒
AI匹配精度	> 98.7%
系统可用性	> 99.9%
T+3结算成功率	> 95%
并发推荐处理	> 1000/秒

10.2 扩展性设计

场景	扩展方案
用户增长	Web/API水平扩展 + Auto Scaling
AI匹配	GPU池化 + 模型优化
区块链	分片 + 多链扩容
结算	消息队列削峰 + 多副本

附录

A. 术语对照表

术语	全称	说明
CCA	Candidate Credit Archive	候选人信用档案
DID	Decentralized Identifier	去中心化身份标识
RRP	RenrenLie Point	平台积分
TrustScore	猎头信任评分	0-100分
HC	Hiring Candidate	招聘需求
KYC	Know Your Customer	实名认证
Overseas HC	出海岗位	海外人才需求
SME	Small and Medium Enterprises	中小企业
T+3	T+3结算	3天快速结算

B. 智能合约地址

合约	网络	地址
Recommend	Fabric	待部署
SplitPayment	Fabric	待部署
TrustScore	Fabric	待部署
RRPRegistry	Fabric	待部署

C. API端点

环境	域名
开发	dev-api.renrenlie.com
测试	test-api.renrenlie.com
生产	api.renrenlie.com

文档状态：v1.8
最后更新：2026年4月30日
审核状态：已完成